The naive Bayes classifier

Alejandro Veloz

Naive Bayes classifier

Naive Bayes is a generative classification algorithm based on Bayes' Theorem.

- **Goal:** Predict the probability of a class y given a set of features $\mathbf{x}$.
- **The "Naive" Assumption:** It assumes that all features x_i are conditionally independent given the class `label`.

(Example) Email Spam Classification:

- Let $y = 1$ (Spam) and $y = 0$ (Ham).
- Features $\mathbf{x} = [x_1, x_2, \dots, x_n]$ represent the presence of words like "Buy" or "Cheap".

The probabilistic framework

Given a feature vector $\mathbf{x} = (x_1, \dots, x_n)$, we want to compute the **posterior probability** for each class C_k:

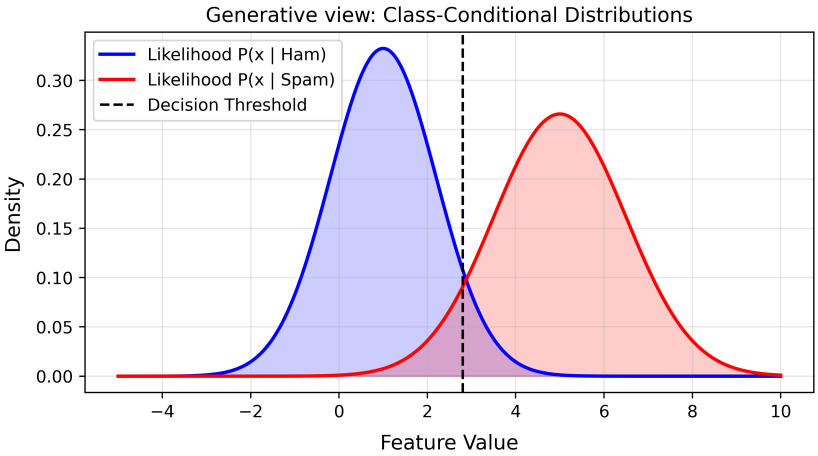
$$P(C_k \mid x_1, \dots, x_n)$$

Using **Bayes' Theorem**:

$$P(C_k \mid \mathbf{x}) = \frac{P(C_k)P(\mathbf{x} \mid C_k)}{P(\mathbf{x})}$$

- $P(C_k)$ is the prior probability (how likely is this class before seeing the data?).
- $P(\mathbf{x} \mid C_k)$ is the likelihood (how well does the class explain the observed features?).
- $P(\mathbf{x})$ is the evidence (normalization constant).

The probabilistic framework



The naive assumption

The "naivety" comes from assuming that features are **mutually independent** given the class C_k:

$$P(x_1, \dots, x_n \mid C_k) = P(x_1 \mid C_k) P(x_2 \mid C_k) \dots P(x_n \mid C_k)$$

This allows us to decompose the likelihood as a product:

$$\boxed{P(\mathbf{x} \mid C_k) = \prod_{i=1}^{n} P(x_i \mid C_k)}$$

Why "Naive"? In practice, words like "Rolex" and "Watch" often appear together. Naive Bayes ignores this correlation to simplify the computation.

Classification decision rule

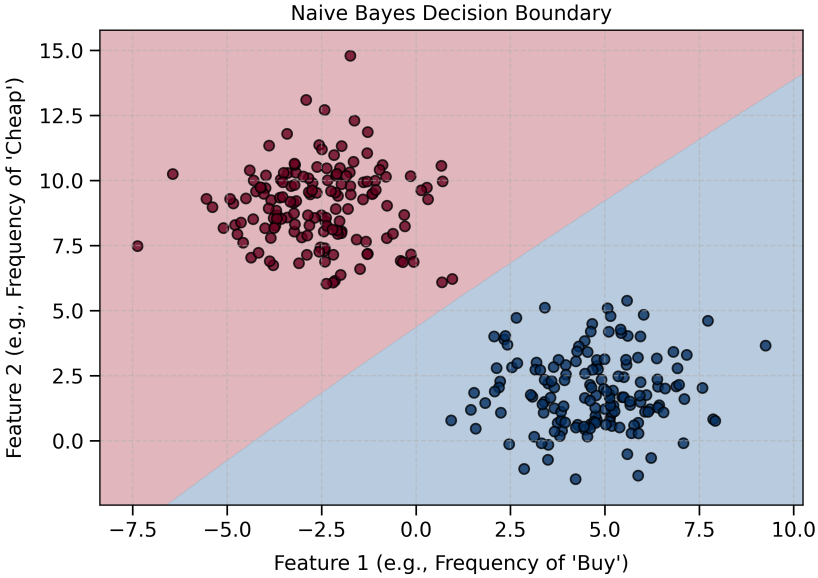
Since the denominator $P(\mathbf{x})$ is constant for all classes, we ignore it for classification. The model becomes:

$$P(C_k \mid x_1, \dots, x_n) \propto P(C_k) \prod_{i=1}^{n} P(x_i \mid C_k)$$

Maximum A Posteriori (MAP) Decision Rule: The classifier chooses the class $\hat{y}$ that maximizes this product:

$$\hat{y} = \underset{k \in \{1,\dots,K\}}{\text{argmax}} \; P(C_k) \prod_{i=1}^{n} P(x_i \mid C_k)$$

Classification decision rule



Parameter estimation (MLE)

We estimate the parameters from the training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ using counts:

Parameter	MLE
Prior $P(C_k)$	$\frac{N_{C_k}}{N_{\text{total}}}$
Likelihood $P(x_i \mid C_k)$	$\frac{N_{x_i, C_k}}{N_{C_k}}$

- N_{C_k}: Number of examples in class k.
- N_{x_i, C_k}: Number of times feature x_i appears in class k.

Example: spam filter

Suppose we have **100 emails**: 25 are Spam (S), 75 are Ham (H).

- **Word "Buy":** appears in 20 Spam and 5 Ham.
- **Word "Cheap":** appears in 15 Spam and 10 Ham.

If a new email arrives with $\mathbf{x} = \{\text{"Buy"}, \text{"Cheap"}\}$:

1. For Spam (S):

$$P(S) \cdot P(\text{"Buy"}|S) \cdot P(\text{"Cheap"}|S) = \frac{25}{100} \cdot \frac{20}{25} \cdot \frac{15}{25} = \mathbf{0.12}$$

2. For Ham (H):

$$P(H) \cdot P(\text{"Buy"}|H) \cdot P(\text{"Cheap"}|H) = \frac{75}{100} \cdot \frac{5}{75} \cdot \frac{10}{75} \approx \mathbf{0.006}$$

Since $0.12 > 0.006$, the prediction is **Spam.**

The zero-frequency problem

If a word (feature) never appeared in the training set for a specific class, then $P(x_i \mid C_k) = 0$.

This zeroes out the entire product.

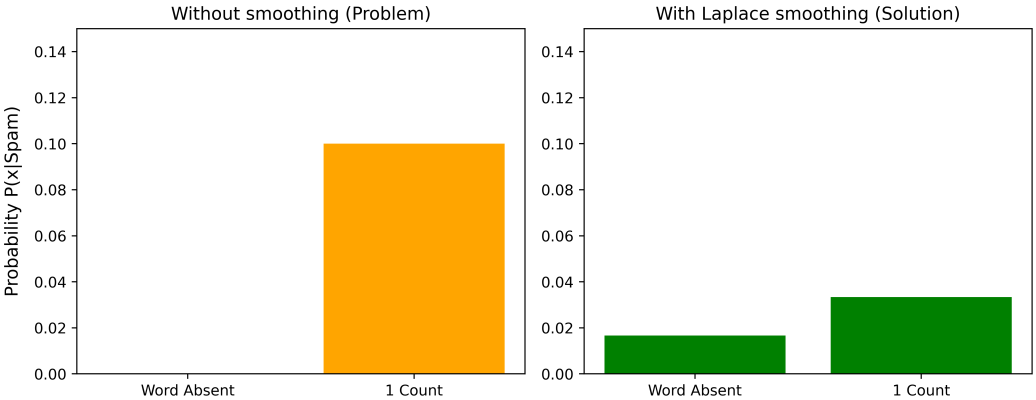
Solution: Laplace smoothing

We add a small smoothing parameter α (usually $\alpha = 1$):

$$\hat{P}(x_i \mid C_k) = \frac{N_{x_i, C_k} + \alpha}{N_{C_k} + \alpha d}$$

where d is the total number of unique features (vocabulary size).

Laplace smoothing



Model variants for text

The choice of distribution for $P(x_i \mid C_k)$ depends on the feature type:

- **Bernoulli NB:** Used when features are binary (word exists or not).
- **Multinomial NB:** Used for word counts (frequency of terms). Standard for long documents.
- **Gaussian NB:** Used for continuous features (assumes a Normal distribution).

- **Generative Model:** Unlike Logistic Regression (which models $P(y|x)$ directly), Naive Bayes models the joint distribution $P(x, y)$.
- **Efficiency:** Training is extremely fast ($O(N \cdot n)$) as it only requires a single pass to count frequencies.
- **Robustness:** Surprisingly effective for high-dimensional text data even if the independence assumption is violated.